

Seminar 3

Object-Oriented Design, IV1350

Tousif Dewan

`tsdewan@kth.se`

May 3, 2026

Project Members:

Tousif Dewan, `tsdewan@kth.se`

Declaration:

By submitting this assignment, it is hereby declared that I have contributed to the solution alone. It is also declared that I fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1 Introduction

The task for this seminar is to translate the design produced in Seminar 2 for the *Repair Electric Bike* scenario into a working Java program, and to verify the resulting program with unit tests. The work is divided into two tasks. Task 1 is the implementation itself: the basic flow and the startup sequence must be implemented in Java without using exceptions, the user interface may be replaced by a placeholder `View` class with hard-coded calls to the controller, and the program must follow all guidelines presented in Chapter 6 of *A First Course in Object Oriented Development*. Task 2 is to write unit tests that demonstrate an understanding of how to test object-oriented code; for the highest grade, every class in the `controller`, `model`, and `integration` layers must be covered, except classes containing only constructors and getters.

The solution must preserve the qualities required of the design in Seminar 2: high cohesion, low coupling, good encapsulation, and a well-defined public interface. The implementation must also be compilable, executable, and accompanied by a public Git

repository, a sample run printout, and a written report. The remainder of this report describes how the program and the tests were produced in the method section, what the resulting solution looks like under result section.

2 Method

The work was performed in two phases. The first phase translated the Seminar 2 design into Java code; the second phase wrote unit tests for the resulting code.

2.1 Implementing the Program

The starting point was the design class diagram, the startup sequence diagram, and the two operation-level sequence diagrams (receptionist and technician) produced in Seminar 2. Each layer in the design was mapped to a Java package under the namespace `se.kth.iv1350.repairelectricbike`: the five layers `startup`, `view`, `controller`, `model`, and `integration` appear as five sub-packages. The mapping was performed one class at a time, following the system operations as listed on the Controller in the design class diagram.

The implementation followed the guidelines in Chapter 6 of the textbook. Method names were chosen to read as verbs in the imperative form (`findCustomer`, `addRepairTask`, `acceptRepairOrder`); class names were nouns in singular form; package names were short and lowercase; and identifiers were never abbreviated. Each public declaration received a Javadoc comment describing its role; comments inside method bodies were avoided in favour of self-documenting code. The build was managed with Apache Maven, which also fetched the JUnit 5 dependency for the test phase.

Two design decisions deserve explicit mention. First, the project was developed without using exceptions, since exception handling is the topic of Seminar 4. The not-found cases (an unknown phone number, an unknown order identifier) were therefore handled by returning `null` and silently ignoring follow-up calls in the controller. This is intentionally a placeholder; in Seminar 4 these will be replaced with checked exceptions. Second, the design from Seminar 2 was preserved in the implementation. Peer-review feedback received on the Seminar 2 diagrams pointed out that the layers were not visually grouped into packages on the class diagram; this concern is addressed in the implementation through the explicit Java package structure, which makes layer membership unambiguous at the source-code level.

A small renaming was applied during implementation: the startup execution method shown on the Seminar 2 sequence diagram as `Execution()` was implemented as `execution()` so that the identifier follows the Java camelCase convention required by Chapter 6. The semantics are unchanged.

2.2 Writing the Unit Tests

The unit tests were written after the program was complete and a manual end-to-end run via the `View` placeholder had confirmed correct behaviour. The testing framework

used is JUnit 5 (Jupiter), invoked through the Maven Surefire plugin. The test directory mirrors the production directory: every test class lives in the same package as its system under test, but under `src/test/java` rather than `src/main/java`.

The choice of *which* tests to write was guided by three principles drawn from the assessment criteria. The first principle was *one test class per non-trivial production class*. Classes that only store and return values (the four DTOs and the `OrderState` enum) were excluded, as the assignment permits. The eighth excluded class is `Printer`, whose only public method writes to `System.out`; testing such methods is also explicitly excluded by the assignment text. The second principle was *cover every branch of every if-statement*. Each `if` in the production code was identified, and tests were written for both the true and false branches. The third principle was *interesting parameter values rather than trivial getters*: tests probe edge cases such as zero amounts, unknown identifiers, exact-versus-near phone-number matches, and the empty registry, in addition to the happy path.

Each test method was given a descriptive name that reads as a sentence (`addDiagnosticResultRecordsFindingOnCorrectOrder`, `findCustomerReturnsNullWhenNumberDoesNotExist`). Every assertion is paired with a failure message so that a broken test produces a clear diagnostic rather than a bare `AssertionError`. A small fluent test pattern was used throughout: a fresh `RegistryCreator` is constructed in `@BeforeEach` so that each test runs against an isolated, empty registry; this prevents cross-test contamination without the need for explicit teardown logic.

3 Result

3.1 Program

The resulting program implements the basic flow of the *Repair Electric Bike* scenario together with its startup sequence. The full source code is published in a public Git repository at <https://github.com/Tousif2021/repair-electric-bike>.

The project is laid out as a Maven project with the structure shown in Listing 1. There are 14 production classes distributed across the five layers, plus one enum (`OrderState`) in the model layer.

`Main` instantiates a `RegistryCreator`, a `Printer`, a `Controller`, and a `View`, then transfers control to the view by calling `view.execution()`. The view contains hard-coded calls that simulate the receptionist and technician scenarios, and it prints every value returned by the controller. The controller exposes the nine system operations identified during Seminar 2: `findCustomer`, `createRepairOrder`, `printRepairOrder`, `findAllRepairOrders`, `addDiagnosticResult`, `addRepairTask`, `updateRepairOrder`, `acceptRepairOrder`, and `rejectRepairOrder`. Persistence is in-memory: the integration layer stores customers and repair orders in plain Java collections rather than calling out to a real database. Following the Seminar 2 design, `RepairOrder` is responsible for its own persistence (it calls `registry.save(this)` after every modification) and for its own printout (it accepts a `Printer` reference); this prevents the `Controller` from acquiring associations to every other class in the system.

```
src/main/java/se/kth/iv1350/repairelectricbike/  
  startup/      Main.java  
  view/         View.java  
  controller/   Controller.java  
  model/        Amount.java, OrderState.java, DiagnosticReport.java,  
                RepairTask.java, RepairOrder.java  
  integration/  CustomerDTO.java, BikeDTO.java, RepairTaskDTO.java,  
                RepairOrderDTO.java, CustomerRegistry.java,  
                RepairOrderRegistry.java, RegistryCreator.java,  
                Printer.java
```

Figure 1: Production source-tree layout. Each sub-directory under `repairelectricbike` corresponds to one architectural layer.

The complete printout produced by a sample run of the program is shown in Listing 2. It demonstrates: the receptionist locating a known customer and failing to locate an unknown one (the alternative path required by Task 1); the creation and initial printout of a repair order; the technician retrieving all stored orders, recording two diagnostic findings, adding two repair tasks, and updating the order; and finally the customer accepting the work. The state of the order is visible at every print step: `CREATED` after creation, `READY_FOR_APPROVAL` after `updateRepairOrder`, and `ACCEPTED` after `acceptRepairOrder`. The total cost is recomputed from the repair tasks and reflects the 3 500 SEK battery replacement plus the 450 SEK charger-port repair.

3.2 Tests

The test suite contains **65 unit tests across 8 test classes**, executed by JUnit 5 via Maven Surefire. The number of tests per class and the corresponding system under test are summarised in Table 3.

A complete test run produces the summary line `Tests run: 65, Failures: 0, Errors: 0, Skipped: 0` followed by `BUILD SUCCESS`. The test sources live in `src/test/java` under exactly the same package structure as the production code; the full set is also part of the public Git repository linked above.

4 Discussion

This evaluates the solution against six criteria selected from `assessment-criteria-seminar3.pdf`. Three apply to the program (Task 1) and three apply to the tests (Task 2). Each subsection grounds its evaluation in concrete examples from the source code.

4.1 Code Conventions, Naming, and Readability

All identifiers in the program follow the Java conventions described in Chapter 6 of the textbook. Classes are nouns in PascalCase (`RepairOrder`, `CustomerRegistry`, `RegistryCreator`); methods are verb phrases in camelCase (`findCustomer`, `addDiagnosticResult`, `acceptRepairOrder`); fields and parameters are lowercase camelCase; constants are UPPER_SNAKE_CASE (`OrderState.READY_FOR_APPROVAL`); and packages are short, all-lowercase names that match the architectural layers. There are no abbreviated identifiers: `regCreator` is the only short form in the codebase, and it appears as a parameter name to the `Controller` constructor only because using the full name twice in the same line was judged to add noise without clarifying meaning.

The Chapter 6 rule of one explanatory comment per public declaration is followed throughout. Every public class, every public constructor, every public method, and every enum constant has a Javadoc comment that explains *what* the declaration represents rather than *how* it is implemented. Conversely, comments inside method bodies were avoided. The single exception is the body of `RepairOrderRegistry.save`, which contains a one-line comment explaining that the method is intentionally a no-op because storage is in-memory; without this note a reader could mistake the empty body for a forgotten implementation. No method exceeds twenty lines, no parameter list contains more than three parameters, and no class exposes a primitive where an object would carry more meaning: monetary values are wrapped in `Amount`, and the lifecycle of a repair order is expressed as the `OrderState` enum rather than as integer constants or strings.

4.2 MVC and Layer Patterns

The Model–View–Controller pattern and the layered architecture from Seminar 2 are visible in the source tree itself: each of the five sub-packages of `se.kth.iv1350.repairelectricbike` corresponds to exactly one layer. Dependencies between layers run only downward. `Main` (in `startup`) creates the `Controller` and the `View` but does not refer to any model or integration class beyond the `RegistryCreator` it must instantiate. `View` depends on `Controller` and on a small number of integration DTOs that act as carriers between layers; it does not import anything from the model layer. `Controller` delegates all business logic to the `model` package and all data access to the `integration` package. `Model` classes are unaware of the controller and the view. There are no upward dependencies anywhere in the project.

The single controller class is consistent with the textbook's recommendation: it holds the nine system operations identified in Seminar 1 and Seminar 2, and each method delegates to one or two collaborators rather than implementing logic itself. The placeholder `View` also matches the assignment requirement, both in its hard-coded pattern and in its discipline of printing every value returned by the controller.

4.3 Cohesion, Coupling, and Encapsulation

Each class in the implementation has a single, focused responsibility. `RepairOrder` knows about the lifecycle of one repair order; `CustomerRegistry` knows how to look up customers; `Printer` knows how to format and print an order; `Controller` knows how to coordinate calls. Cohesion is therefore high at the class level. The same is true at the layer level: model classes never speak directly to integration classes other than through DTOs, and integration classes never import from the controller layer.

Coupling is kept low through three deliberate choices, all carried over from the Seminar 2 design. First, `RepairOrder` is responsible for its own persistence: after every modification it calls `registry.save(this)` from inside the model class itself, rather than asking the controller to make that call. The controller therefore needs no association to `RepairOrderRegistry` for write operations, and the textbook “spider-in-the-web” anti-pattern is avoided. Second, the printout is also delegated downward: `Controller.printRepairOrder` looks up the order and calls `order.printRepairOrder(printer)`, so the controller never holds a direct dependency to the `Printer` class for the act of formatting. Third, `RegistryCreator` hides the existence of individual registries from the startup layer; `Main` depends on `RegistryCreator`, not on `CustomerRegistry` and `RepairOrderRegistry` separately, so adding a new registry in the future would be a one-line change in `Main` rather than a rewrite.

Encapsulation is enforced both by visibility modifiers and by immutability. The constructors of `CustomerRegistry` and `RepairOrderRegistry` are package-private, which means the only legitimate way to obtain a registry is to ask `RegistryCreator` for one; this prevents higher layers from constructing parallel registries that escape the application’s data model. All four DTOs (`CustomerDTO`, `BikeDTO`, `RepairTaskDTO`, `RepairOrderDTO`) are declared `final` with all-`final` fields and no setters; once a DTO has been handed to the view, the view cannot mutate it and accidentally corrupt the model. `RepairOrder.getRepairTasks` returns an unmodifiable view of the task list, so callers can iterate but cannot insert tasks behind the model’s back. Finally, state transitions go through the `setState` method on `RepairOrder` rather than through direct field assignment; if validation rules need to be added later (for example, forbidding the transition `CREATED` → `ACCEPTED`), they can be placed in a single method without ripple-effect changes elsewhere.

4.4 JUnit Framework Usage

The tests are written using JUnit 5 (Jupiter), which is the testing framework used in the textbook and lectures. Methods are annotated with `@Test` and `@BeforeEach`; assertions come from `org.junit.jupiter.api.Assertions`, including `assertEquals`, `assertNotNull`, `assertNull`, `assertTrue`, `assertFalse`, `assertSame`, `assertNotSame`, `assertThrows`, and `assertDoesNotThrow`. The tests are executed automatically by Maven Surefire on every `mvn test` invocation; no manual test runner is needed.

Every assertion in the suite carries a failure message. The messages explain in business terms why the assertion is expected to hold, for example: *“updateRepairOrder should*

transition the state to READY_FOR_APPROVAL.” or “The repair task list must be unmodifiable from outside.” A passing test produces no console output of its own; only Surefire’s per-class summary is printed. A failing test would produce its own descriptive message, the assertion’s expected and actual values, and a stack trace pointing to the failing line. The tests are therefore self-evaluating in the sense required by the assessment criteria.

4.5 Branch Coverage of If-statements

A deliberate effort was made to cover both branches of every `if` statement in the production code. Eleven `if` statements appear across the model, integration, and controller layers (excluding the two in `View`, whose helper methods only format text for printing and are thus not under test). Each one has a corresponding test for the true branch and a corresponding test for the false branch.

Three concrete examples illustrate the pattern. In `Controller`, the seven system operations that look up an order before acting on it follow the shape “`if (order != null) ...`” so that an unknown order identifier produces no side effect. The true branch is exercised by tests such as `addDiagnosticResultRecordsFindingOnCorrectOrder`; the false branch is exercised by tests such as `addDiagnosticResultOnUnknownIdIsSilentlyIgnored`, which use `assertDoesNotThrow` to confirm that no exception escapes when the identifier does not exist. In `Amount.equals`, the `instanceof` check has both an equal-types case (`equalsReturnsTrueForEqualValues`) and a wrong-type case (`equalsReturnsFalseForNonAmountObject`, which passes a `String` and a `null`). In `CustomerRegistry.findCustomer`, the loop’s body contains an equality test on the phone number; this is covered by `findCustomerReturnsCustomerWhenNumberExists` (match), `findCustomerReturnsNullWhenNumberDoesNotExist` (no match in any iteration), and `findCustomerMatchesExactlyOnPhoneNumber` (a near-match that should not be returned). Tests were designed to cover both branches of each identified condition.

4.6 Test Completeness for Distinction

For the highest grade, the assessment criteria require unit tests for every class in the controller, model, and integration layers, except classes whose only members are constructors that store values and getters that return them, and except methods that only produce output to `System.out`. The submitted suite meets this requirement. Every class with non-trivial behaviour has a dedicated test class living in the matching package: `ControllerTest` for the controller; `AmountTest`, `DiagnosticReportTest`, `RepairTaskTest`, and `RepairOrderTest` for the model; and `CustomerRegistryTest`, `RepairOrderRegistryTest`, and `RegistryCreatorTest` for the integration layer. Eight production classes are intentionally not tested: the four DTOs and the `OrderState` enum (no behaviour beyond storage), the `Printer` (the only public method writes to `System.out` and is excluded by the assignment), and the two startup-layer classes `Main` and `View` (which are not part of the controller, model, or integration layers).

The tests probe more than the happy path. `RepairOrderTest` alone contains fourteen tests covering construction, three state transitions, the empty-task and multi-task cases for `getTotalCost`, the immutability of the returned task list, and a full lifecycle test that walks an order from `CREATED` through `ACCEPTED`. `RepairOrderRegistryTest` verifies that a new registry is empty, that order identifiers increment monotonically, that `findRepairOrder` returns `null` for an unknown identifier, and that `save` on an existing order does not silently remove it. `ControllerTest`, the largest of the test classes, contains a dedicated test for each of the nine system operations, plus negative tests for unknown identifiers, plus a single end-to-end scenario test (`fullScenarioEndsInAcceptedStateWithCorrectTotal`) that drives the controller through the entire receptionist–technician–customer flow and asserts that the final state is `ACCEPTED` and the final total is 3950 SEK. This approach reduces robustness and will be replaced with explicit exception handling in Seminar 4.

References

- Lindbäck, L. *A First Course in Object Oriented Development*, Chapters 6–7. Course textbook, IV1350.
- Course material from IV1350, Object-Oriented Design, KTH (lecture slides and lecture *Practice Programming and Unit Testing*).
- JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
- Apache Maven Documentation. <https://maven.apache.org/>
- Source code repository for this submission. <https://github.com/Tousif2021/repair-electric-bike>


```

>> Receptionist looks up customer by phone number 070-1234567
Controller returned: Tousif Dewan <tsdewan@kth.se> 070-1234567

>> Receptionist looks up customer by phone number 000-0000000
Controller returned: null (customer not found)

>> Receptionist registers a new repair order
Controller returned: Order #1 [CREATED] Battery will not charge (total: 0.00)

>> Receptionist prints the repair order

===== REPAIR ORDER =====
Order ID:      1
State:         CREATED
Description:    Battery will not charge
--- Customer ---
Tousif Dewan
tsdewan@kth.se
070-1234567
--- Bike ---
Kawasaki Pro Mountain Bike
Serial no:     0012
Total cost:    0.00
=====

>> Technician retrieves all repair orders
Controller returned 1 order(s):
- Order #1 [CREATED] Battery will not charge (total: 0.00)

>> Technician records diagnostic findings
>> Technician adds repair tasks
>> Technician updates and finalizes the order
>> Final printout before customer decision

===== REPAIR ORDER =====
Order ID:      1
State:         READY_FOR_APPROVAL
Description:    Replace battery pack and repair loose charger port
--- Customer ---
Tousif Dewan
tsdewan@kth.se
070-1234567
--- Bike ---
Kawasaki Pro Mountain Bike
Serial no:     0012
Total cost:    3950.00
=====

>> Customer accepts the repair order
>> Final printout after acceptance

===== REPAIR ORDER =====
Order ID:      1
State:         ACCEPTED
Description:    Replace battery pack and repair loose charger port
--- Customer ---
Tousif Dewan
tsdewan@kth.se
070-1234567
--- Bike ---
Kawasaki Pro Mountain Bike
Serial no:     0012
Total cost:    3950.00
=====

```

Figure 2: Sample run output produced⁹⁽¹⁰⁾ by `mvn exec:java`. The state transitions `CREATED` $\rightarrow$ `READY_FOR_APPROVAL` $\rightarrow$ `ACCEPTED` are visible across the three printouts.

Layer	Test class	Tests	System under test
controller	ControllerTest	19	Controller
model	AmountTest	9	Amount
model	DiagnosticReportTest	4	DiagnosticReport
model	RepairTaskTest	3	RepairTask
model	RepairOrderTest	14	RepairOrder
integration	CustomerRegistryTest	4	CustomerRegistry
integration	RepairOrderRegistryTest	7	RepairOrderRegistry
integration	RegistryCreatorTest	5	RegistryCreator
Total		65	

Figure 3: Distribution of unit tests across the test classes. Every non-trivial production class in the `controller`, `model`, and `integration` layers has exactly one corresponding test class.